

Claims Loss-Cause Journey — Jupyter Notebook Code

Complete copy/paste version of the business-analysis notebook. Code is rendered in a monospaced font with preserved indentation and line breaks.

Usage: Copy each code block into a separate Jupyter Notebook code cell. Markdown sections are included as headings/context.

Claims Loss-Cause Journey — Business Analysis Notebook

****Purpose:**** Run this notebook against your claims CSV and automatically produce business-ready tables, graphs, KPI summaries, and presentation-ready PNGs.

Required business views

1. Overall
2. Property – Catastrophe (`NEW\_SEGMENT = Property` + `CAT\_IND = Y`)
3. Property – Non-Catastrophe (`NEW\_SEGMENT = Property` + `CAT\_IND = N`)
4. Non-Injury (`NEW\_SEGMENT = Non-Injury`)

Business questions answered

- How long is the journey from FNOL to the latest information?
- How long does a claim spend in each loss cause?
- What are the most common first → last loss-cause journeys?
- How does the portfolio break down by Line of Business?
- How often does a cause change co-occur with a segment transfer?
- How does claim behaviour vary by Contact Method?
- Do repeated cause changes correspond to longer journeys / more segment transfers?

****Important:**** the notebook uses only the fields supplied. Cause-change and segment-transfer analysis is reported as association/co-occurrence, not causal proof.

Code Cell 1

```
# =====  
# 0. USER SETTINGS – CHANGE ONLY THIS SECTION  
# =====  
CSV_PATH = "claims_data.csv"  
  
# Folder where presentation-ready outputs will be saved  
OUTPUT_DIR = "claims_business_outputs"  
  
# Number of categories shown in Top-N charts
```

```

TOP_N = 15

# If your contact-method column uses a different separator, add it here.
CONTACT_SPLIT_REGEX = r"[>,;|/]+"

# Whether to save every chart as a high-resolution PNG
SAVE_PNG = True

# =====
# Expected columns
# =====
EXPECTED_COLUMNS = [
    "LOSS_SK",
    "LINE_OF_BUSINESS",
    "CAT_IND",
    "NEW_SEGMENT",
    "CONTACT_METHOD",
    "CAUSE_CHANGE_COUNT",
    "SEGMENT_SEQUENCE",
    "SEGMENT_TRANSFER_COUNT",
    "FNOL_EFF_DIFF",
    "CAUSE_DAYS_DISTRIBUTION"
]

```

Code Cell 2

```

# =====
# 1. IMPORTS
# =====
import os
import re
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from IPython.display import display, Markdown

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 100)
pd.set_option("display.max_rows", 100)
pd.set_option("display.float_format", lambda x: f"{x:,.2f}")

sns.set_theme(style="whitegrid")
os.makedirs(OUTPUT_DIR, exist_ok=True)

print("Libraries loaded.")

```

Code Cell 3

```

# =====
# 2. LOAD + VALIDATE CSV
# =====
df = pd.read_csv(CSV_PATH)

missing = [c for c in EXPECTED_COLUMNS if c not in df.columns]
if missing:
    raise ValueError(
        "The CSV is missing these required columns:\n"
    )

```

```

        + "\n".join(f"- {c}" for c in missing)
    )

df = df[EXPECTED_COLUMNS].copy()

# Clean text
for c in [
    "LOSS_SK", "LINE_OF_BUSINESS", "CAT_IND",
    "NEW_SEGMENT", "CONTACT_METHOD",
    "SEGMENT_SEQUENCE", "CAUSE_DAYS_DISTRIBUTION"
]:
    df[c] = df[c].astype("string").str.strip()

df["CAT_IND"] = df["CAT_IND"].str.upper()
df["NEW_SEGMENT"] = (
    df["NEW_SEGMENT"]
    .str.replace(r"\s+", " ", regex=True)
    .str.strip()
)

# Numeric conversion
for c in [
    "CAUSE_CHANGE_COUNT",
    "SEGMENT_TRANSFER_COUNT",
    "FNOL_EFF_DIFF"
]:
    df[c] = pd.to_numeric(df[c], errors="coerce")

print(f"Rows: {len(df):,}")
print(f"Unique claims: {df['LOSS_SK'].nunique(dropna=True):,}")

display(df.head())

```

Code Cell 4

```

# =====
# 3. DATA QUALITY CHECK
# =====
quality = pd.DataFrame({
    "Column": df.columns,
    "Missing Count": [df[c].isna().sum() for c in df.columns],
    "Missing %": [df[c].isna().mean() * 100 for c in df.columns],
    "Unique Values": [df[c].nunique(dropna=True) for c in df.columns]
})

display(quality.sort_values("Missing %", ascending=False).round(2))

print("Duplicate LOSS_SK rows:", df["LOSS_SK"].duplicated().sum())

```

Code Cell 5

```

# =====
# 4. CREATE THE FOUR BUSINESS VIEWS
# =====
def business_view(row):
    seg = str(row["NEW_SEGMENT"]).strip().lower() if pd.notna(row["NEW_SEGMENT"]) else ""
    cat = str(row["CAT_IND"]).strip().upper() if pd.notna(row["CAT_IND"]) else ""

    if seg == "property" and cat == "Y":
        return "Property - Catastrophe"

```

```

    if seg == "property" and cat == "N":
        return "Property - Non-Catastrophe"
    if seg == "non-injury":
        return "Non-Injury"
    return "Other / Unmapped"

df["BUSINESS_VIEW"] = df.apply(business_view, axis=1)

VIEWS = [
    "Overall",
    "Property - Catastrophe",
    "Property - Non-Catastrophe",
    "Non-Injury"
]

VIEW_DF = {
    "Overall": df.copy(),
    "Property - Catastrophe": df[df["BUSINESS_VIEW"] == "Property - Catastrophe"].copy(),
    "Property - Non-Catastrophe": df[df["BUSINESS_VIEW"] == "Property - Non-Catastrophe"].copy(),
    "Non-Injury": df[df["BUSINESS_VIEW"] == "Non-Injury"].copy()
}

population = pd.DataFrame({
    "View": VIEWS,
    "Claims": [VIEW_DF[v]["LOSS_SK"].nunique() for v in VIEWS]
})

population["Share %"] = population["Claims"] / population.loc[0, "Claims"] * 100

display(population.round(2))

```

A. Claim journey duration — FNOL to latest information

Code Cell 6

```

# =====
# 5. FNOL JOURNEY METRICS
# =====
def fnol_metrics(d):
    x = d["FNOL_EFF_DIFF"].dropna()
    if len(x) == 0:
        return pd.Series({
            "Claims": 0, "Mean Days": np.nan, "Median Days": np.nan,
            "P75 Days": np.nan, "P90 Days": np.nan, "P95 Days": np.nan,
            "Max Days": np.nan
        })

    return pd.Series({
        "Claims": len(x),
        "Mean Days": x.mean(),
        "Median Days": x.median(),
        "P75 Days": x.quantile(.75),
        "P90 Days": x.quantile(.90),
        "P95 Days": x.quantile(.95),
        "Max Days": x.max()
    })

FNOL_SUMMARY = pd.DataFrame({v: fnol_metrics(VIEW_DF[v]) for v in VIEWS}).T
display(FNOL_SUMMARY.round(2))

# Business buckets

```

```

FNOL_BINS = [-np.inf, 7, 30, 60, 90, 180, np.inf]
FNOL_LABELS = ["0–7 days", "8–30 days", "31–60 days", "61–90 days", "91–180 days", "181+ days"]

def fnol_bucket_table(d):
    x = d["FNOL_EFF_DIFF"].dropna()
    buckets = pd.cut(x, bins=FNOL_BINS, labels=FNOL_LABELS)
    out = buckets.value_counts(sort=False).rename("Claims").to_frame()
    out["Share %"] = out["Claims"] / len(x) * 100 if len(x) else np.nan
    return out

display(fnol_bucket_table(VIEW_DF["Overall"]).round(2))

```

Code Cell 7

```

# =====
# 6. PRESENTATION CHART – FNOL JOURNEY BY BUSINESS VIEW
# =====
plot_df = df[df["BUSINESS_VIEW"].isin(VIEWS[1:]).copy()]

plt.figure(figsize=(12, 6))
sns.boxplot(
    data=plot_df,
    x="BUSINESS_VIEW",
    y="FNOL_EFF_DIFF",
    showfliers=False
)
plt.title("FNOL to Latest Information – Claim Journey Duration")
plt.xlabel("")
plt.ylabel("Days")
plt.xticks(rotation=20)
plt.tight_layout()

if SAVE_PNG:
    plt.savefig(
        os.path.join(OUTPUT_DIR, "01_FNOL_journey_duration.png"),
        dpi=300,
        bbox_inches="tight"
    )
plt.show()

```

B. Loss-cause journey — duration + first-to-last movement

Code Cell 8

```

# =====
# 7. PARSE CAUSE_DAYS_DISTRIBUTION
# =====
def parse_cause_days(value):
    if pd.isna(value) or str(value).strip() == "":
        return []

    output = []
    for item in str(value).split(">"):
        item = item.strip()
        if ":" not in item:
            continue

        cause, days = item.rsplit(":", 1)
        cause = cause.strip()

```

```

    try:
        days = float(days.strip())
    except ValueError:
        continue

    if cause:
        output.append((cause, days))

return output

def cause_features(value):
    parsed = parse_cause_days(value)

    if not parsed:
        return pd.Series({
            "FIRST_LOSS_CAUSE": pd.NA,
            "LAST_LOSS_CAUSE": pd.NA,
            "TOTAL_CAUSE_DAYS": np.nan,
            "CAUSE_STATE_COUNT": 0,
            "CAUSE_TRANSITION_COUNT": 0,
            "AVG_DAYS_PER_CAUSE_STATE": np.nan,
            "MAX_DAYS_ONE_CAUSE_STATE": np.nan,
            "FIRST_TO_LAST_CAUSE": pd.NA
        })

    causes = [x[0] for x in parsed]
    days = [x[1] for x in parsed]

    return pd.Series({
        "FIRST_LOSS_CAUSE": causes[0],
        "LAST_LOSS_CAUSE": causes[-1],
        "TOTAL_CAUSE_DAYS": sum(days),
        "CAUSE_STATE_COUNT": len(causes),
        "CAUSE_TRANSITION_COUNT": max(len(causes) - 1, 0),
        "AVG_DAYS_PER_CAUSE_STATE": np.mean(days),
        "MAX_DAYS_ONE_CAUSE_STATE": max(days),
        "FIRST_TO_LAST_CAUSE": f"{causes[0]} → {causes[-1]}"
    })

features = df["CAUSE_DAYS_DISTRIBUTION"].apply(cause_features)
df = pd.concat([df, features], axis=1)

# Refresh views
VIEW_DF = {
    "Overall": df.copy(),
    "Property - Catastrophe": df[df["BUSINESS_VIEW"] == "Property - Catastrophe"].copy(),
    "Property - Non-Catastrophe": df[df["BUSINESS_VIEW"] == "Property - Non-Catastrophe"].copy(),
    "Non-Injury": df[df["BUSINESS_VIEW"] == "Non-Injury"].copy()
}

display(df[[
    "LOSS_SK", "CAUSE_DAYS_DISTRIBUTION",
    "FIRST_LOSS_CAUSE", "LAST_LOSS_CAUSE",
    "TOTAL_CAUSE_DAYS", "CAUSE_STATE_COUNT",
    "CAUSE_TRANSITION_COUNT", "AVG_DAYS_PER_CAUSE_STATE",
    "MAX_DAYS_ONE_CAUSE_STATE"
]].head(10))

```

Code Cell 9

```

# =====
# 8. CAUSE-DURATION SUMMARY
# =====
CAUSE_DURATION_SUMMARY = pd.DataFrame({
    v: {
        "Claims": len(VIEW_DF[v]),
        "Avg Total Cause Days": VIEW_DF[v]["TOTAL_CAUSE_DAYS"].mean(),
        "Median Total Cause Days": VIEW_DF[v]["TOTAL_CAUSE_DAYS"].median(),
        "P90 Total Cause Days": VIEW_DF[v]["TOTAL_CAUSE_DAYS"].quantile(.90),
        "Avg Days per Cause State": VIEW_DF[v]["AVG_DAYS_PER_CAUSE_STATE"].mean(),
        "Avg Cause States": VIEW_DF[v]["CAUSE_STATE_COUNT"].mean(),
        "Avg Cause Transitions": VIEW_DF[v]["CAUSE_TRANSITION_COUNT"].mean(),
        "Avg Longest Cause Stay": VIEW_DF[v]["MAX_DAYS_ONE_CAUSE_STATE"].mean()
    }
    for v in VIEWS
}).T

display(CAUSE_DURATION_SUMMARY.round(2))

def explode_cause_days(d):
    rows = []
    for _, r in d[["LOSS_SK", "CAUSE_DAYS_DISTRIBUTION"]].iterrows():
        for cause, days in parse_cause_days(r["CAUSE_DAYS_DISTRIBUTION"]):
            rows.append({
                "LOSS_SK": r["LOSS_SK"],
                "CAUSE": cause,
                "DAYS": days
            })
    return pd.DataFrame(rows)

CAUSE_EVENTS = explode_cause_days(df)

CAUSE_SUMMARY = (
    CAUSE_EVENTS
    .groupby("CAUSE")
    .agg(
        Claim_States=("LOSS_SK", "count"),
        Unique_Claims=("LOSS_SK", "nunique"),
        Total_Cause_Days=("DAYS", "sum"),
        Avg_Days=("DAYS", "mean"),
        Median_Days=("DAYS", "median")
    )
    .reset_index()
    .sort_values("Total_Cause_Days", ascending=False)
)

display(CAUSE_SUMMARY.head(TOP_N).round(2))

```

Code Cell 10

```

# =====
# 9. PRESENTATION CHART – WHERE TIME IS SPENT
# =====
temp = CAUSE_SUMMARY.head(TOP_N).sort_values("Total_Cause_Days")

plt.figure(figsize=(11, 7))
sns.barplot(
    data=temp,
    y="CAUSE",

```

```

        x="Total_Cause_Days"
    )
    plt.title("Top Loss Causes by Total Recorded Cause-Days")
    plt.xlabel("Total Recorded Cause-Days")
    plt.ylabel("Loss Cause")
    plt.tight_layout()

    if SAVE_PNG:
        plt.savefig(
            os.path.join(OUTPUT_DIR, "02_loss_cause_total_days.png"),
            dpi=300,
            bbox_inches="tight"
        )
    plt.show()

```

Code Cell 11

```

# =====
# 10. FIRST CAUSE → LAST CAUSE HEATMAPS
# =====
def first_last_matrix(d, top_n=TOP_N):
    x = d.dropna(subset=["FIRST_LOSS_CAUSE", "LAST_LOSS_CAUSE"]).copy()

    common = pd.concat([
        x["FIRST_LOSS_CAUSE"],
        x["LAST_LOSS_CAUSE"]
    ]).value_counts().head(top_n).index

    x = x[
        x["FIRST_LOSS_CAUSE"].isin(common) &
        x["LAST_LOSS_CAUSE"].isin(common)
    ]

    return pd.crosstab(
        x["FIRST_LOSS_CAUSE"],
        x["LAST_LOSS_CAUSE"]
    )

for view in VIEWS:
    matrix = first_last_matrix(VIEW_DF[view])

    if matrix.empty:
        print(f"No usable cause journey data for {view}")
        continue

    plt.figure(figsize=(12, 9))
    sns.heatmap(matrix, annot=True, fmt="g", cmap="Blues")
    plt.title(f"First Loss Cause → Last Loss Cause | {view}")
    plt.xlabel("Last Loss Cause")
    plt.ylabel("First Loss Cause")
    plt.tight_layout()

    if SAVE_PNG:
        safe = re.sub(r"^[A-Za-z0-9]+", "_", view).strip("_")
        plt.savefig(
            os.path.join(OUTPUT_DIR, f"03_heatmap_{safe}.png"),
            dpi=300,
            bbox_inches="tight"
        )

```



```
plt.show()
```

Code Cell 12

```
# =====
# 11. TOP FIRST → LAST JOURNEYS
# =====
FLOW_TABLES = {}

for view in VIEWS:
    flow = (
        VIEW_DF[view]
        .dropna(subset=["FIRST_TO_LAST_CAUSE"])
        .groupby("FIRST_TO_LAST_CAUSE")
        .size()
        .reset_index(name="Claims")
        .sort_values("Claims", ascending=False)
    )

    FLOW_TABLES[view] = flow

print(f"TOP FIRST → LAST LOSS-CAUSE JOURNEYS – {view}")
display(flow.head(15))
```

C. Line of Business — volume + journey behaviour

Code Cell 13

```
# =====
# 12. LINE OF BUSINESS BREAKDOWN
# =====
def lob_table(d):
    x = d.copy()
    x["CAUSE_CHANGED"] = x["CAUSE_CHANGE_COUNT"].fillna(0) > 0
    x["SEGMENT_TRANSFERRED"] = x["SEGMENT_TRANSFER_COUNT"].fillna(0) > 0

    out = (
        x.groupby("LINE_OF_BUSINESS", dropna=False)
        .agg(
            Claims=("LOSS_SK", "nunique"),
            Median_FNOL_Days=("FNOL_EFF_DIFF", "median"),
            Avg_FNOL_Days=("FNOL_EFF_DIFF", "mean"),
            Avg_Total_Cause_Days=("TOTAL_CAUSE_DAYS", "mean"),
            Avg_Cause_Changes=("CAUSE_CHANGE_COUNT", "mean"),
            Avg_Segment_Transfers=("SEGMENT_TRANSFER_COUNT", "mean"),
            Cause_Change_Rate=("CAUSE_CHANGED", "mean"),
            Segment_Transfer_Rate=("SEGMENT_TRANSFERRED", "mean")
        )
        .reset_index()
    )

    out["Share %"] = out["Claims"] / out["Claims"].sum() * 100
    out["Cause_Change_Rate"] *= 100
    out["Segment_Transfer_Rate"] *= 100

    return out.sort_values("Claims", ascending=False)

LOB_TABLES = {}
```

```

for view in VIEWS:
    LOB_TABLES[view] = lob_table(VIEW_DF[view])

    print(f"LINE OF BUSINESS - {view}")
    display(LOB_TABLES[view].head(25).round(2))

```

Code Cell 14

```

# =====
# 13. PRESENTATION CHART - LOB VOLUME
# =====
lob_plot = LOB_TABLES["Overall"].head(TOP_N).sort_values("Claims")

plt.figure(figsize=(11, 7))
sns.barplot(
    data=lob_plot,
    y="LINE_OF_BUSINESS",
    x="Claims"
)
plt.title("Claim Volume by Line of Business - Overall")
plt.xlabel("Unique Claims")
plt.ylabel("Line of Business")
plt.tight_layout()

if SAVE_PNG:
    plt.savefig(
        os.path.join(OUTPUT_DIR, "04_LOB_claim_volume.png"),
        dpi=300,
        bbox_inches="tight"
    )
plt.show()

```

D. Cause change ↔ segment transfer

Code Cell 15

```

# =====
# 14. CAUSE CHANGE VS SEGMENT TRANSFER
# =====
def trigger_analysis(d):
    x = d.copy()

    x["CAUSE_CHANGED"] = x["CAUSE_CHANGE_COUNT"].fillna(0) > 0
    x["SEGMENT_CHANGED"] = x["SEGMENT_TRANSFER_COUNT"].fillna(0) > 0

    cause_n = x["CAUSE_CHANGED"].sum()
    segment_n = x["SEGMENT_CHANGED"].sum()
    both_n = (x["CAUSE_CHANGED"] & x["SEGMENT_CHANGED"]).sum()

    return pd.Series({
        "Claims": len(x),
        "Cause-change cases": cause_n,
        "Segment-transfer cases": segment_n,
        "Both": both_n,
        "Cause-change → segment-transfer %":
            both_n / cause_n * 100 if cause_n else np.nan,
        "Segment-transfer → cause-change %":
            both_n / segment_n * 100 if segment_n else np.nan,
        "Both as % of all claims":

```

```

        both_n / len(x) * 100 if len(x) else np.nan
    })

TRIGGER_SUMMARY = pd.DataFrame({
    view: trigger_analysis(VIEW_DF[view])
    for view in VIEWS
}).T

display(TRIGGER_SUMMARY.round(2))

# Overall 2x2 matrix
x = VIEW_DF["Overall"].copy()
x["CAUSE_CHANGE_STATUS"] = np.where(
    x["CAUSE_CHANGE_COUNT"].fillna(0) > 0,
    "Cause Change",
    "No Cause Change"
)
x["SEGMENT_TRANSFER_STATUS"] = np.where(
    x["SEGMENT_TRANSFER_COUNT"].fillna(0) > 0,
    "Segment Transfer",
    "No Segment Transfer"
)

MATRIX = pd.crosstab(
    x["CAUSE_CHANGE_STATUS"],
    x["SEGMENT_TRANSFER_STATUS"]
)

display(MATRIX)

```

Code Cell 16

```

# =====
# 15. PRESENTATION CHART – CAUSE CHANGE → SEGMENT TRANSFER
# =====
trigger_plot = TRIGGER_SUMMARY.reset_index().rename(columns={"index": "View"})

plt.figure(figsize=(11, 6))
sns.barplot(
    data=trigger_plot.iloc[1:],
    x="View",
    y="Cause-change → segment-transfer %"
)
plt.title("Cause-Change Cases Also Having a Segment Transfer")
plt.xlabel("")
plt.ylabel("Share of Cause-Change Cases (%)")
plt.xticks(rotation=20)
plt.tight_layout()

if SAVE_PNG:
    plt.savefig(
        os.path.join(OUTPUT_DIR, "05_cause_change_segment_transfer.png"),
        dpi=300,
        bbox_inches="tight"
    )
plt.show()

```

E. Contact Method segmentation

Code Cell 17

```
# =====
# 16. CONTACT METHOD
# =====
def explode_contacts(d):
    x = d[["LOSS_SK", "CONTACT_METHOD"]].copy()

    x["CONTACT_METHOD_EXPLODED"] = (
        x["CONTACT_METHOD"]
        .fillna("UNKNOWN")
        .astype(str)
        .apply(lambda s: [
            z.strip().upper()
            for z in re.split(CONTACT_SPLIT_REGEX, s)
            if z.strip()
        ])
    )

    return x.explode("CONTACT_METHOD_EXPLODED")

def contact_table(d):
    base = d.copy()
    base["CAUSE_CHANGED"] = base["CAUSE_CHANGE_COUNT"].fillna(0) > 0
    base["SEGMENT_TRANSFERRED"] = base["SEGMENT_TRANSFER_COUNT"].fillna(0) > 0

    contacts = explode_contacts(base)

    out = (
        contacts.merge(
            base[
                [
                    "LOSS_SK", "FNOL_EFF_DIFF", "TOTAL_CAUSE_DAYS",
                    "CAUSE_CHANGED", "SEGMENT_TRANSFERRED"
                ]
            ],
            on="LOSS_SK",
            how="left"
        )
        .groupby("CONTACT_METHOD_EXPLODED")
        .agg(
            Unique_Claims=("LOSS_SK", "nunique"),
            Median_FNOL_Days=("FNOL_EFF_DIFF", "median"),
            Avg_Total_Cause_Days=("TOTAL_CAUSE_DAYS", "mean"),
            Cause_Change_Rate=("CAUSE_CHANGED", "mean"),
            Segment_Transfer_Rate=("SEGMENT_TRANSFERRED", "mean")
        )
        .reset_index()
    )

    out["Cause_Change_Rate"] *= 100
    out["Segment_Transfer_Rate"] *= 100

    return out.sort_values("Unique_Claims", ascending=False)

CONTACT_TABLES = {}

for view in VIEWS:
    CONTACT_TABLES[view] = contact_table(VIEW_DF[view])
```

```
print(f"CONTACT METHOD - {view}")
display(CONTACT_TABLES[view].head(20).round(2))
```

Code Cell 18

```
# =====
# 17. PRESENTATION CHART - CONTACT METHOD
# =====
contact_plot = CONTACT_TABLES["Overall"].head(TOP_N).sort_values("Unique_Claims")

plt.figure(figsize=(11, 7))
sns.barplot(
    data=contact_plot,
    y="CONTACT_METHOD_EXPLODED",
    x="Unique_Claims"
)
plt.title("Claim Volume by Contact Method - Overall")
plt.xlabel("Unique Claims")
plt.ylabel("Contact Method")
plt.tight_layout()

if SAVE_PNG:
    plt.savefig(
        os.path.join(OUTPUT_DIR, "06_contact_method_volume.png"),
        dpi=300,
        bbox_inches="tight"
    )
plt.show()
```

F. Cause-change intensity

Code Cell 19

```
# =====
# 18. CAUSE-CHANGE INTENSITY
# =====
def cause_change_intensity(d):
    x = d.copy()

    x["CAUSE_CHANGE_BUCKET"] = pd.cut(
        x["CAUSE_CHANGE_COUNT"].fillna(0),
        bins=[-1, 0, 1, 2, 3, np.inf],
        labels=["0", "1", "2", "3", "4+"]
    )

    x["SEGMENT_TRANSFERRED"] = x["SEGMENT_TRANSFER_COUN"].fillna(0) > 0

    out = (
        x.groupby("CAUSE_CHANGE_BUCKET", observed=False)
        .agg(
            Claims=("LOSS_SK", "nunique"),
            Median_FNOL_Days=("FNOL_EFF_DIFF", "median"),
            Median_Total_Cause_Days=("TOTAL_CAUSE_DAYS", "median"),
            Avg_Segment_Transfers=("SEGMENT_TRANSFER_COUN", "mean"),
            Segment_Transfer_Rate=("SEGMENT_TRANSFERRED", "mean")
        )
        .reset_index()
    )
```

```

    out["Segment_Transfer_Rate"] *= 100
    return out

INTENSITY_TABLES = {}

for view in VIEWS:
    INTENSITY_TABLES[view] = cause_change_intensity(VIEW_DF[view])

    print(f"CAUSE-CHANGE INTENSITY - {view}")
    display(INTENSITY_TABLES[view].round(2))

```

Code Cell 20

```

# =====
# 19. PRESENTATION CHART - CAUSE-CHANGE INTENSITY VS JOURNEY
# =====
temp = INTENSITY_TABLES["Overall"].copy()

plt.figure(figsize=(10, 6))
sns.barplot(
    data=temp,
    x="CAUSE_CHANGE_BUCKET",
    y="Median_FNOL_Days"
)
plt.title("Median FNOL → Latest Duration by Number of Cause Changes")
plt.xlabel("Cause-Change Count")
plt.ylabel("Median Days")
plt.tight_layout()

if SAVE_PNG:
    plt.savefig(
        os.path.join(OUTPUT_DIR, "07_cause_change_intensity.png"),
        dpi=300,
        bbox_inches="tight"
    )
plt.show()

```

G. Automated business summary

Code Cell 21

```

# =====
# 20. AUTOMATIC BUSINESS SUMMARY
# =====
def safe_pct(a, b):
    return (a / b * 100) if b else np.nan

overall = VIEW_DF["Overall"]

cause_changed = overall["CAUSE_CHANGE_COUNT"].fillna(0) > 0
segment_changed = overall["SEGMENT_TRANSFER_COUNT"].fillna(0) > 0
both = cause_changed & segment_changed

top_lob = LOB_TABLES["Overall"].iloc[0] if len(LOB_TABLES["Overall"]) else None
top_cause = CAUSE_SUMMARY.iloc[0] if len(CAUSE_SUMMARY) else None
top_flow = FLOW_TABLES["Overall"].iloc[0] if len(FLOW_TABLES["Overall"]) else None
top_contact = CONTACT_TABLES["Overall"].iloc[0] if len(CONTACT_TABLES["Overall"]) else None

summary_lines = [

```

```

f"• The dataset contains {overall['LOSS_SK'].nunique():,} unique claims.",
f"• Median FNOL → latest duration is {overall['FNOL_EFF_DIFF'].median():,.1f} days; the P90 is {overall['FNOL_EFF_DIFF'].quantile(.90):,.1f} days.",
f"• {safe_pct(cause_changed.sum(), len(overall)):.1f}% of claims have at least one recorded loss-cause change.",
f"• {safe_pct(segment_changed.sum(), len(overall)):.1f}% of claims have at least one recorded segment transfer.",
f"• Among claims with a cause change, {safe_pct(both.sum(), cause_changed.sum():.1f}% also have a segment transfer.",
]

if top_lob is not None:
    summary_lines.append(
        f"• The highest-volume Line of Business is {top_lob['LINE_OF_BUSINESS']} "
        f"with {int(top_lob['Claims']):,} claims ({top_lob['Share %']:.1f}% of claims)."
    )

if top_cause is not None:
    summary_lines.append(
        f"• {top_cause['CAUSE']} accounts for the largest amount of recorded cause-days "
        f"({top_cause['Total_Cause_Days']:.0f} days across cause states)."
    )

if top_flow is not None:
    summary_lines.append(
        f"• The most frequent first → last cause journey is {top_flow['FIRST_TO_LAST_CAUSE']} "
        f"({int(top_flow['Claims']):,} claims)."
    )

if top_contact is not None:
    summary_lines.append(
        f"• The most common contact method is {top_contact['CONTACT_METHOD_EXPLODED']} "
        f"({int(top_contact['Unique_Claims']):,} unique claims)."
    )

display(Markdown(
    "### Presentation-ready observations\n\n" +
    "\n".join(summary_lines)
))

with open(os.path.join(OUTPUT_DIR, "business_summary.txt"), "w", encoding="utf-8") as f:
    f.write("\n".join(summary_lines))

```

H. Export all business tables to Excel

Code Cell 22

```

# =====
# 21. EXPORT EXCEL PACK
# =====
excel_path = os.path.join(OUTPUT_DIR, "claims_business_analysis.xlsx")

with pd.ExcelWriter(excel_path, engine="openpyxl") as writer:
    population.to_excel(writer, sheet_name="Population", index=False)
    quality.to_excel(writer, sheet_name="Data_Quality", index=False)
    FNOL_SUMMARY.to_excel(writer, sheet_name="FNOL_Summary")
    CAUSE_DURATION_SUMMARY.to_excel(writer, sheet_name="Cause_Duration")
    CAUSE_SUMMARY.to_excel(writer, sheet_name="Cause_Summary", index=False)
    TRIGGER_SUMMARY.to_excel(writer, sheet_name="Cause_Segment")
    LOB_TABLES["Overall"].to_excel(writer, sheet_name="LOB_Overall", index=False)
    CONTACT_TABLES["Overall"].to_excel(writer, sheet_name="Contact_Overall", index=False)
    INTENSITY_TABLES["Overall"].to_excel(writer, sheet_name="Cause_Intensity", index=False)
    FLOW_TABLES["Overall"].to_excel(writer, sheet_name="Top_Cause_Flows", index=False)

```

```
print(f"Excel pack created: {excel_path}")
print(f"PNG charts saved in: {OUTPUT_DIR}")
```

How to use this notebook for your business presentation

Step 1

Put your CSV in the same folder as the notebook.

Step 2

Change only:

```
```python
CSV_PATH = "your_file_name.csv"
```
```

Step 3

Run ****Run All****.

Step 4

Use the generated outputs:

- `01\_FNOL\_journey\_duration.png`
- `02\_loss\_cause\_total\_days.png`
- `03\_heatmap\_\*.png`
- `04\_LOB\_claim\_volume.png`
- `05\_cause\_change\_segment\_transfer.png`
- `06\_contact\_method\_volume.png`
- `07\_cause\_change\_intensity.png`

These are saved at ****300 DPI****, so they can be inserted directly into PowerPoint.

Suggested presentation storyline

****Slide 1 — Executive overview****

- Claim population
- Median and P90 FNOL journey
- Cause-change rate
- Segment-transfer rate
- Cause-change → segment-transfer rate

****Slide 2 — Claim journey duration****

- FNOL journey distribution
- Median vs P90
- Four business populations

****Slide 3 — Loss-cause movement****

- First → last cause heatmap
- Top cause journeys
- Total cause-days

****Slide 4 — Line of Business****

- Claim volume
- Median journey duration
- Cause-change / segment-transfer behaviour

****Slide 5 — Cause change and segment movement****

- 2×2 matrix
- Cause-change → segment-transfer rate

****Slide 6 — Contact method****

- Volume by contact method
- Cause-change and segment-transfer behaviour

****Slide 7 — Key observations****

- Use the automatically generated business summary as the starting point for your commentary.

Interpretation guardrails

- `CAUSE\_DAYS\_DISTRIBUTION` describes recorded cause states and durations; repeated causes are retained as separate states.
- `FIRST → LAST` is a journey summary, not a complete transition graph.
- Cause-change and segment-transfer are treated as **co-occurring events**, not proof that one caused the other.
- Contact methods are exploded when multiple methods are stored in one claim, so contact-method claim counts can overlap.